

Argus Security Analysis Report

Repository: https://github.com/arnav-sutraway/Vuln_test

Total Findings: 21

1. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 22

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

2. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: auth.py, line 39

Evidence: + "" WHERE email = ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

3. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 11

Evidence: query = "SELECT * FROM users WHERE username = " + username + ""

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

4. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 24

Evidence: query = f"SELECT * FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

5. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 39

Evidence: `f"WHERE username = '{username}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

6. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: database.py, line 74

Evidence: query = f"SELECT * FROM products ORDER BY {sort_column}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

7. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: High

Confidence: 88.0%

Location: admin.py, line 55

Evidence: query = f"SELECT * FROM audit_logs WHERE action LIKE '%{search_term}%'"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

8. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 10

Evidence: "WHERE email = " + email + " "

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

9. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/users.py, line 24

Evidence: query = f"DELETE FROM users WHERE id = {user_id}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

10. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 10

Evidence: `f"WHERE type = '{report_type}' "`

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

11. Potential SQL Injection

Vulnerability Type: SQL Injection

Severity: Medium

Confidence: 72.0%

Location: services/reports.py, line 24

Evidence: query = f"SELECT * FROM {table_name}"

Description: The code appears to construct SQL statements using dynamic string content. If user-controlled input reaches this expression, an attacker may modify the intended SQL query.

Recommendation: Use parameterized queries or prepared statements. Do not concatenate or interpolate untrusted input into SQL statements.

12. Hardcoded API Key

Vulnerability Type: Hardcoded Secret

Severity: Medium

Confidence: 90.0%

Location: auth.py, line 9

Evidence: API_KEY = "ARGUS_FAKE_API...[REDACTED]"

Description: An API key credential is hardcoded directly in the module rather than being retrieved from environment variables or a secret store.

Recommendation: Remove the hardcoded API key from source code and load it dynamically at runtime using environment variables or a secret management service.

13. Hardcoded Database Password

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: auth.py, line 13

Evidence: DATABASE_PASSWORD = "[REDACTED]"

Description: A database authentication password is hardcoded directly in the source file, exposing sensitive database access credentials.

Recommendation: Remove the hardcoded password and supply it via secure runtime configuration, such as environment variables or a vault.

14. Hardcoded JWT Secret Key

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: auth.py, line 51

Evidence: JWT_SECRET = "[REDACTED]"

Description: A secret used for session token generation and signing is statically embedded in the generate_session_token function, enabling unauthorized token generation if exposed.

Recommendation: Retrieve the JWT signing secret from a secure secret management service or environment variable.

15. Hardcoded Stripe Secret Key

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 7

Evidence: STRIPE_SECRET_KEY = "sk_test_[REDACTED]"

Description: A Stripe API secret key is defined directly in the configuration file, risking unauthorized access to payment processor operations.

Recommendation: Store payment provider API keys in a secret management system or environment variables, and ensure the credential is rotated.

16. Hardcoded AWS Access Key ID

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 10

Evidence: AWS_ACCESS_KEY_ID = "AKIA[REDACTED]"

Description: An AWS Access Key identifier is statically defined in configuration, paired with a secret access key.

Recommendation: Use IAM roles, AWS Secrets Manager, or environment variables to provide AWS credentials dynamically.

17. Hardcoded AWS Secret Access Key

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 13

Evidence: `AWS_SECRET_ACCESS_KEY = "[REDACTED]"`

Description: An AWS Secret Access Key is embedded directly in configuration code, which can allow unauthorized access to cloud resources.

Recommendation: Remove the secret access key from code, rotate the key in IAM, and utilize runtime configuration or IAM instance profiles.

18. Hardcoded Database Password

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: config.py, line 16

Evidence: DATABASE_PASSWORD = "[REDACTED]"

Description: A database password is hardcoded directly in the configuration module.

Recommendation: Inject the database password at runtime using environment variables or a secrets manager.

19. Hardcoded GitHub Personal Access Token

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 19

Evidence: GITHUB_TOKEN = "ghp_[REDACTED]"

Description: A GitHub personal access token with a standard ghp_ prefix is embedded in configuration code.

Recommendation: Revoke the exposed token, generate a replacement, and inject it via environment variables or secret vaults.

20. Hardcoded Encryption Key

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 90.0%

Location: config.py, line 22

Evidence: ENCRYPTION_KEY = "[REDACTED]"

Description: A cryptographic encryption key is statically assigned in configuration, jeopardizing data confidentiality.

Recommendation: Supply encryption keys through secure key management systems (e.g., KMS) rather than embedding them in source files.

21. Hardcoded Database Credentials in Connection String

Vulnerability Type: Hardcoded Secret

Severity: High

Confidence: 95.0%

Location: config.py, line 25

Evidence: DATABASE_URL = "postgresql://testuser:[REDACTED]@localhost/testdb"

Description: A database connection URI containing an embedded username and password is hardcoded in the application configuration.

Recommendation: Construct connection strings dynamically from environment variables or load the entire URI from a secrets manager.